Automatic Number Plate Recognition

An Image Processing & Computer Vision pipeline with context-aware OCR disambiguation

OpenCV • EasyOCR • FastAPI • React

Course: Image Processing & Computer Vision (IPCV)

Presented by: Varun Kumar Sahu (20233357)
Anubhav Vishwakarma (20231015)
Vivekanand Pandey (20233318)
Vivek Kumar (20233317)

Why plain OCR fails on number plates

1 Camera angle & tilt

Even 5°–15° of tilt stops text detectors from segmenting horizontal lines.

2 2-line plates

Bikes and SUVs stack State+District over Series+Number; raw OCR returns them out of order.

3 Look-alike characters

Q/O/0, G/6, I/1, B/8, S/5, Z/2 — and HSRP's open-top 4 is read as “L”.

4 Extra plate elements

Blue IND strip, black bezel read as [ ], and rivets read as dots or “B”.

5 Lighting & shadows

Headlight glare and uneven shadows break global thresholding.

Result

Tesseract and stock EasyOCR work on flat scanned documents — not on real-world vehicle plates.

Objectives and solution overview

What the system does

1. **Detect & localize**
   Find the plate in a still image or live webcam feed.

2. **Straighten & enhance**
   Auto-deskew tilts up to ±35°, then denoise and equalize contrast.

3. **Read accurately**
   Multi-pass OCR, 2-line ordering, and plate-syntax correction.

4. **Export & integrate**
   FastAPI REST endpoints, Swagger UI, and Excel export.

Tech stack

- OpenCV 4.x — image processing
- EasyOCR — text recognition
- FastAPI — backend API
- React 18 + Vite — web UI
- openpyxl — Excel export

Two usage modes

- Upload Image — inspect boxes, crops, enhanced output
- Live Camera — ~15–30 FPS tracking

System architecture

Frontend
React 18 + Vite

- Upload Image mode
- Live Camera (WebRTC)
- Base64 / multipart frames

Backend
FastAPI + OpenCV + EasyOCR

- Plate detection
- IPCV preprocessing
- Multi-variant OCR
- Clustering & disambiguation

Output
JSON response

- Bounding boxes + text + confidence
- Annotated image & plate crops
- HSRP-style badge, live HUD

Client sends frames to FastAPI (port 8000); results return as JSON and are rendered in the browser.

The IPCV pipeline: 7 stages

1	2	3	4	5	6	7
Localization	**Deskewing**	**Upscaling**	**Denoise + CLAHE**	**Multi-pass OCR**	**Clustering**	**Slot rules**
Haar cascade	Hough lines + affine	2× bicubic	Bilateral + local contrast	EasyOCR, 3 variants	Lines & border stripping	Context-aware disambiguation

Pixel domain — image processing (stages 1–4)

Recognition & text domain (stages 5–7)

Input image → verified plate text, e.g. MH 46 X 9996

Core image-processing techniques

Technique	OpenCV / module	Parameters	Purpose
Haar cascade detection	CascadeClassifier	haarcascade_russian_plate_number.xml	Locate candidate plate ROIs
Dynamic padding + full-crop override	detect_plates	5% W, 8% H; aspect 2.0–6.5	Avoid clipping edge characters
Auto-deskewing	HoughLinesP, warpAffine	median edge angle, ±1.5° to ±35°	Straighten tilted plates
Bicubic upscaling	resize, INTER_CUBIC	min height ≥ 160 px	Keep loops of 0 / 8 separate
Bilateral filter	bilateralFilter	d=9, σColor=75, σSpace=75	Denoise, keep edges sharp
CLAHE	createCLAHE	clipLimit=2.5, tiles 8×8	Fix shadows and glare locally
Otsu binarization	threshold + OTSU	3rd OCR variant	Boost recall on hard plates

Context-aware disambiguation engine

DL	01	AB	1234
Slot 1 · State (letters)	Slot 2 · District (digits)	Slot 3 · Series (letters)	Slot 4 · Number (digits)

Numeric slots: letter → digit

- Q, O, D, U → 0 I,] → 1
- L → 4 (HSRP open-top 4)
- Z → 2 S → 5 G, b → 6 B → 8

`MH L6 → MH 46` `B571 → 8571`

Alphabetic slots: digit → letter + dictionary

- 0 → Q 1 → I 5 → S 8 → B etc.
- State code checked against 36 valid codes
- OL → DL, HH → MH, TM → TN, 6J → GJ

`DL 7C0 1939 → DL 7C Q 1939`

Results and benchmark

7 / 7

benchmark samples passed

82%

confidence, 14° tilted plate

Condition	Before	After (pipeline)
2-line plate	Misordered	**DL 01 AB 1234**
14° tilt + 2-line	Tilt failure	**MH 12 GJ 1264**
Bezel + rivet dots	HR 26 CKB 5711	**HR 26 CK 8571**
HSRP open-top 4	MH 16 X 9996	**MH 46 X 9996**
Adjacent zeros	KA 64 NO 0991	**KA 64 N 0099**
Delhi category C	DL 70 Q 1939	**DL 7C Q 1939**
Left IND strip	JND TNBTC 5106	**TN 87 C 5106**

Tested on real-world mobile camera photos and edge cases.

Conclusion and future scope

What we achieved

- End-to-end ANPR for images and live video
- Deskew, denoise and CLAHE make OCR robust to tilt and lighting
- Positional rules fix look-alike characters
- All 7 benchmark cases read correctly

Future scope

- Replace the Haar cascade with a learned detector (e.g. YOLO)
- Evaluate on a larger, labelled plate dataset
- Track plates across video frames

Thank you — Questions?